CO2 Controller Plan

Sep 28, 2026 · @Lauren

1. The plan in one page

CO_2 is controlled by a PI controller tuned with lambda tuning (λ = 200 s, Kp = 0.0313, Ki = 1.83×10^{-5} per percentage point), updated every 15 s from the ExplorIR-M sensor, with the duty turned into one solenoid pulse per 15 s window; in simulation it fills from room air to 5% in about 9 min with 0.06 percentage points (pp) of overshoot.

Updated for the ExplorIR-M (T90 < 30 s, 2 readings per second). The sensor is about four times faster than the 60 s lag the first version assumed, so the loop is now faster (λ 300 → 200 s) and updates more often (T_s 30 → 15 s), which halves the ripple.

Key findings:

- **Don't fill with the valve held fully open.** At 0.1 L/min the chamber fills from room air to 5% in only 30 s. Even with the fast ExplorIR-M, a sensor-triggered full-open fill overshoots by about 2 pp, and that takes around 45 min to leak away. The PI alone fills gently (starting duty about 16%).
- **Use b = 1 here, not 0.5.** The plant (29 min) is much slower than the loop (3 min), so setpoint weighting leaves a very slow tail.
- **Order the 0 to 20% version of the ExplorIR-M, and turn off its auto-zero.** The auto-zero assumes the sensor regularly sees fresh air (400 ppm). In an incubator held at 5% it never does, so it would slowly recalibrate itself wrong (section 5).
- **Mixing now matters more than the sensor.** With a fast sensor, the slowest part of the measurement is CO_2 spreading from the inlet to the sensor. Keep the two apart and let the air circulate.

The order to work in:

- [ ] Tidy up the model write-up (section 2).
- [ ] Choose the control period and pulse scheme for the solenoid (section 3).
- [ ] Design the PI with lambda tuning (section 4).
- [ ] Configure the ExplorIR-M: range, auto-zero, filter (section 5).
- [ ] Run the simulations: fill, lid opening, robustness (section 6).
- [ ] Check stability and the discrete loop (section 7).
- [ ] Build it in Simulink (section 8).
- [ ] Write the firmware logic (section 9).

- [ ] Once built: leak test, fill test, retune (section 10).

2. The CO_2 model

Your model and numbers are correct: the balance, the linearisation, $\mu_0 = 1.72\%$, $\tau = 1711$ s and $K = 2.73$ all check out.

The mass balance

```
V \frac{d\chi_c}{dt} = \mu_v Q_{in} (\chi_s - \chi_c) - Q_L (\chi_c - \chi_a)
```

In words: when the valve is open, CO_2-rich gas flows in and pushes the same volume of chamber gas out, so the net gain is Q_in(χ_s − χ_c). Separately, leaks swap chamber gas for room air at Q_L. This assumes the gas is well mixed and the pressure stays at atmospheric.03

Symbol	Meaning	Value	Check
V	Free gas volume, 213 × 155 × 30 mm	0.99 L	✓
χ_s, χ_a, χ_0	Supply, room, target CO_2 fraction	1.0, 0.0004, 0.05	✓
Q_in	Flow with valve open	0.1 L/min = 1.667×10^{-3} L/s	✓ (limit 0.625 L/min)
Q_L	Leak exchange flow, 2 air changes per hour	5.5×10^{-4} L/s	✓ (assumption)
μ_0	Duty to hold 5%	1.72%	✓

The transfer function

Working in percentage points (pp) of CO_2 rather than fractions, which is easier to read:

```
G(s) = \frac{\delta\chi(s)}{\delta\mu(s)} = \frac{K}{\tau s + 1} = \frac{273.6}{1711s + 1} \quad [\text{pp per unit duty}]
```

- **K = 273.6 pp:** the valve fully open would drive the chamber towards 100% CO_2. Holding 5% needs only 1.7% duty. The actuator is massively oversized for holding, which is why the valve size was chosen for resolution, not capacity.
- **τ = 1711 s (29 min):** almost entirely the leak. Without the valve, CO_2 decays with time constant V/Q_L = 1800 s.

- **Full-open fill time:** 0.04% to 5% takes only **30 s** with the valve fully open.

How CO_2 differs from temperature

- **You can only add CO_2, never remove it.** Just like the heaters, but worse: an overshoot only goes away through leaks, with a 30 min time constant. A 0.4 pp overshoot takes about 40 min to clear.
- **The actuator is fast, the process is slow, and the sensor sits in between.** The chamber can change in seconds when the valve opens, but the plant's natural recovery takes half an hour. The sensor's response lag (section 5) ends up setting the loop speed.
- **The actuator is on/off.** A solenoid is either open or closed, so "duty" has to be made from timed pulses (section 3).

Small changes worth making to the write-up

- **The nonlinearity is mild.** The gain depends on $(\chi_s - \chi_c)$, which only changes from 1.0 to 0.95 between room air and 5%. The $\mu_0 Q_{in}$ term is only 5% of the τ denominator. So the linear model is a good description all the way from room air to 5%, not only near 5%. That means the PI can do the fill too, and the separate full-open fill strategy isn't needed (and is actually harmful, section 6).
- **Say what "well mixed" relies on.** CO_2 diffusing across a 10 to 20 cm chamber with no air movement takes around 10 min. Put the inlet and the sensor apart, add a small diffuser at the inlet, and ideally let the heater convection or a small fan stir the air. Otherwise the sensor reads a local puff rather than the chamber average.
- **The fill-test formula ignores leaks.** That's fine because the test lasts seconds and the leak time constant is 30 min, but say so.
- **Mention the sensor range** (section 5), since it is a hard requirement.

3. The solenoid: turning duty into pulses

Every 15 s the controller computes a duty u, and the valve opens once for $u \times 15$ s, rounded to the nearest 0.1 s; this gives 0.67% duty resolution and about 0.06 pp of ripple.

Time-proportioning

A solenoid is on or off, so "duty" means the fraction of each control window the valve is open. This is called time-proportioning control:

$$t_{on} = \text{round}\left(\frac{u \cdot T_s}{t_{min}}\right) t_{min}, \qquad t_{on} < t_{min} \Rightarrow t_{on} = 0$$

At the operating point, u = 1.72%, so each 15 s window has a pulse of about 0.2 to 0.3 s. One minimum pulse (0.1 s) adds 0.016 pp of CO_2, well inside your ±0.1 pp target.

Choosing the control period T_s

T_s is a trade-off. Short windows give coarse duty steps, because the 0.1 s minimum is a big fraction of a short window. Long windows give big individual pulses and more ripple. A faster sensor also allows a shorter window, since there's no point updating much faster than the measurement responds. Simulated fill from room air, λ = 200 s, ExplorIR-M (T90 < 30 s):

T_s	Duty resolution	Overshoot	Settles (±0.2 pp)	Ripple (peak to peak)
5 s	2% (larger than μ_0)	0.24 pp	34 min (limit cycles)	0.05 pp
10 s	1%	0.10 pp	10 min	0.05 pp
15 s	**0.67%**	**0.06 pp**	**9 min**	**0.06 pp**
30 s	0.33%	0.06 pp	10 min	0.10 pp
60 s	0.17%	0.14 pp	9 min	0.18 pp

T_s = 15 s is the best balance: low ripple, no limit cycling, and T_s = λ/13, inside the λ/10 rule. The ripple is roughly $\mu_0 \times T_s \times 0.16$ pp/s, so it grows with a longer window or a leakier chamber.

Practical points

- **Valve life:** one pulse every 15 s is about 5800 cycles a day, or about 2 million a year. Check the solenoid's rated cycle life; many small gas solenoids are rated for tens of millions, but some cheap ones aren't. If yours is low, use T_s = 30 s (twice the ripple, half the cycles).
- **Minimum pulse:** 0.1 s is your pull-in estimate. Measure it (section 10). If it's longer, increase T_s so the duty resolution t_min/T_s stays below about 1%.
- **Flow during a pulse:** very short pulses may not reach full flow while the tubing pressurises. The fill test measures the effective Q_in, which includes this.

4. The controller: PI with lambda tuning

The CO_2 controller is a PI with Kp = 0.0313 and Ki = 1.83×10^{-5} (error in pp, output as duty), giving a first-order closed loop with λ = 200 s and no setpoint weighting (b = 1).

The method is identical to the chamber and glass loops. PI, not PID, because the plant is first order and a D term would amplify sensor noise.

Step 1: choose λ = 200 s

λ is set by the lags between the gas entering and the controller seeing it:

Lag	Size	Where it comes from
Sensor	about 13 s time constant	ExplorIR-M T90 < 30 s; for a first-order lag, τ ≈ T90/2.3
Sampling	about 7.5 s	Half of the 15 s control window
Averaging readings	about 2.5 s	Averaging the last 5 s of readings (section 9)
Mixing	unknown, 0 to 60 s	CO_2 travelling from the inlet to the sensor

Without mixing, the total is about 25 s, and λ could be as small as about 120 s. But mixing is the unknown. In simulation with a 60 s mixing lag, λ = 150 s overshoots by 0.38 pp, while λ = 200 s stays at 0.11 pp. So λ = 200 s is chosen: fast (5% in about 9 min) and safe even if mixing is slow.

The starting duty from room air is Kp × 4.96 = 0.16, so the valve never has to open fully.

Steps 2 to 4: from λ to the gains

The target closed loop 1/(λs + 1) needs the loop L = 1/(λs). The controller is C = L ÷ G:

```
C(s) = \frac{1}{200s}\cdot\frac{1711s + 1}{273.6} = \frac{1711s + 1}
{54\,720\,s} = 0.0313 + \frac{1.83\times10^{-5}}{s}
```

Parameter	Formula	Value
Kp	τ / (Kλ) = 1711 / (273.6 × 200)	0.0313 (3.1% duty per pp of error)
Ki	1 / (Kλ) = 1 / 54 720	1.83×10^{-5}
Ti	Kp / Ki = τ	1711 s

If you work in ppm (the sensor's native output), divide both gains by 10 000: Kp = 3.13×10^{-6} and Ki = 1.83×10^{-9} per ppm. Converting the reading to percent first is simpler.

Step 5: the loop and closed loop

$$L(s) = \frac{1711s + 1}{54\,720\,s}\cdot\frac{273.6}{1711s + 1} = \frac{1}{200s} \qquad \frac{\chi_c(s)}{\chi_{sp}(s)} = \frac{1}{200s + 1}$$

Why b = 1 here (the opposite of the temperature loops)

With setpoint weighting, the step response is

$$y(t) = 1 - \frac{(1-b)\,\tau}{\tau - \lambda}\,e^{-t/\tau} - \frac{b\tau - \lambda}{\tau - \lambda}\,e^{-t/\lambda}$$

For CO_2, τ = 1711 s is more than eight times λ, so any b < 1 puts a large weight on the very slow e^(−t/1711) term. With b = 0.5 that weight is 0.57, and the last part of the fill crawls in over an hour. With b = 1 the fill overshoot is only 0.06 pp, so there's nothing to fix. For the temperature loops, τ and λ were much closer, so the slow tail was small and b = 0.5 was worth it.

Anti-windup matters at the bottom limit

For temperature the upper clamp mattered. For CO_2 the **lower** one does: after any overshoot, the controller wants a negative duty (to remove CO_2), gets clamped at 0, and without anti-windup the integrator would keep winding down for the whole time the leak takes to clear the overshoot. Then it would undershoot. Use the same clamping anti-windup as the other loops.

5. The sensor: ExplorIR-M

The ExplorIR-M is a good fit (fast, 2 readings per second, ranges up to 100%), but it needs the 0 to 20% range, its auto-zero turned off, and a calibration at 5% to meet the ±0.1 to 0.2 pp target.

Key specs (datasheet)

Spec	Value	What it means for the design
Ranges	0 to 5%, 20%, 60% or 100%	Use **0 to 20%**. The 5% version tops out exactly at the setpoint, so any overshoot would be clipped and invisible
Response time	T90 < 30 s, diffusion limited	First-order time constant about 13 s; sets λ (section 4)

Spec	Value	What it means for the design
Reading rate	2 per second	Plenty; average several readings per control window
Accuracy	±(70 ppm + 5% of reading)	At 5%: ±0.26 pp absolute. Larger than your target, so calibrate (below)
Warm-up	< 1.2 s to first reading	No start-up delay needed
Humidity	0 to 95% RH, non-condensing	Keep it above the dew point
Interface	UART	Read in firmware every 0.5 s
Built-in auto-zero	Yes	**Turn it off** (below)

Response speed

T90 < 30 s means the sensor reaches 90% of a step change within 30 s. For a first-order lag, T90 ≈ 2.3τ, so τ ≈ 13 s. That's much faster than the 60 s lag assumed in the first version of this plan, which is why λ went from 300 to 200 s and T_s from 30 to 15 s.

The quoted T90 is at the sensor's default digital filter setting. The filter trades noise for speed; leave it at the default or lower, because a heavier filter slows the sensor and eats into the phase margin.

Simulated fill from room air with the final design (λ = 200 s, T_s = 15 s):

Case	Overshoot	Settles (±0.2 pp)
ExplorIR-M as specified (T90 = 30 s)	0.06 pp	9 min
Sensor twice as slow (T90 = 60 s)	0.06 pp	9 min
Plus 30 s mixing lag	0.07 pp	8 min
Plus 60 s mixing lag	0.11 pp	5 min
Sensor noise 0.02 pp rms	0.06 pp	10 min

The sensor is now fast enough that it isn't the limiting factor; **mixing** is. Keep the gas inlet and sensor apart, add a small diffuser at the inlet, and let the heater convection (or a small fan) move the air.

Turn off the auto-zero

The built-in auto-zero assumes the sensor regularly sees fresh air (about 400 ppm) and quietly corrects its baseline towards that. In an incubator held at 5%, it never sees fresh air, so over days it would drag the reading wrong and the controller would follow it. The Gas Sensing Solutions user manual lists the UART commands to disable auto-zero and to zero the sensor manually. Use them: disable auto-zero, and zero in fresh air before each experiment with the lid open.

Calibration for accuracy

±0.26 pp at 5% is the absolute accuracy out of the box. The controller will hold the **reading** at 5% very tightly (within about 0.06 pp in simulation), but the real CO_2 could be up to 0.26 pp off. To meet ±0.1 to 0.2 pp:

1. Zero in fresh air (400 ppm) with auto-zero off.
2. Check a span point: flush the chamber with a certified 5% CO_2 calibration gas (common for incubators) and adjust the span, or compare against a calibrated incubator analyser if the lab has one.
3. Do this at 37 °C, since NDIR readings shift with temperature and pressure.

Other sensor points

- **Humidity:** cell-culture chambers are humid. Mount the sensor where it stays warm (the warm glass helps) so its optics don't fog.
- **Placement:** away from the gas inlet, at roughly the plate height, where the cells are.

6. Simulation results

The PI fills cleanly, handles lid openings and copes with large errors in the leak and flow estimates; a complete loss of CO_2 makes the plain PI overshoot about 0.3 pp, which freezing the integrator during big disturbances cuts to under 0.1 pp.

All results use the full nonlinear mass balance, 15 s time-proportioned pulses with a 0.1 s minimum, the ExplorIR-M response (T90 = 30 s) with 5 s averaging, λ = 200 s and b = 1, unless stated.

Filling from room air: don't hold the valve open

Fill strategy	Overshoot	Settles (±0.2 pp)
PI from the start (recommended)	**0.06 pp**	**9 min**

Fill strategy	Overshoot	Settles (±0.2 pp)
Valve fully open until the sensor reads 4%, then PI	2.2 pp	47 min

With the valve fully open, the chamber reaches 5% in 30 s. Even the fast ExplorIR-M is still catching up, and the valve only closes at the end of a 15 s window, so it overshoots by over 2 pp, and that can only leak away. The PI starts at 16% duty and never gets ahead of the sensor.

Lid openings

Starting settled at 5%, then the concentration drops suddenly:

Disturbance	Peak after recovery	Back within ±0.2 pp
Short opening, drops to 4%	+0.12 pp	4 min
Full loss, drops to 1%	+0.33 pp	32 min
Full loss, with a 60 s mixing lag	+0.44 pp	34 min

After a full loss, the integrator builds up during the recovery and pushes about 0.3 pp past 5%. Only the leak can bring it back, with its 30 min time constant. That's the physics of a gas you can only add.

The fix: freeze the integrator during big disturbances. Once the chamber has reached 5% for the first time, the integrator already holds the right steady duty (μ_0), learned from the real chamber. After that, only let it integrate while the error is under 0.5 pp. During a lid-open recovery, P does the work and the integrator stays at its learned value, so nothing winds up:

Full loss (5% → 1%)	Plain PI	PI with integrator freeze
Nominal	+0.33 pp, 33 min	**+0.08 pp, 9 min**
60 s mixing lag	+0.44 pp, 32 min	+0.11 pp, 5 min
Leak 3× estimate	+0.21 pp, 47 min	+0.14 pp, 8 min
Leak 0.5× estimate	+0.32 pp, 27 min	+0.03 pp, 9 min
Flow 0.5× setting	+0.43 pp, 49 min	+0.03 pp, 16 min
Partial loss (5% → 2.5%)	+0.23 pp, 20 min	+0.08 pp, 7 min

The fill from room air is unchanged (the freeze only switches on after the first time the chamber reaches 5%), and because the integrator value is learned rather than calculated from the model, it works across the whole range of leak and flow errors. This goes into the firmware logic (section 9).

Robustness to the unknowns

The controller is tuned for the estimated Q_L and Q_in; the simulated plant uses different values:

Plant case	Overshoot	Settles	Ripple
Nominal	0.06 pp	9 min	0.06 pp
Leak half the estimate (1 air change/h)	0.19 pp	8 min	0.04 pp
Leak three times the estimate (6 air changes/h)	0.05 pp	65 min (slow final creep)	0.15 pp
Flow half the setting (0.05 L/min)	0.02 pp	21 min	0.05 pp
Flow double the setting (0.2 L/min)	0.04 pp	4 min	0.07 pp

Nothing goes unstable. The leaky-chamber case creeps in slowly because the real τ is then about 10 min, much shorter than the 29 min the controller was tuned for, so its integral time is too long. That is exactly what the leak test and retune in section 10 fix.

7. Stability and discrete-time check

The CO_2 loop is stable with about 84° of phase margin including the ExplorIR-M lag and 15 s sampling (about 75° if mixing adds 30 s), and the discrete poles sit inside the unit circle.

Check	Result	What it means
Closed-loop poles (continuous)	s = −1/200 and s = −1/1711	Real and negative, stable
Phase margin, ideal	90°	L(s) = 1/(200s)
Lost to sensor lag (13 s)	atan(13/200) = 3.7°	At crossover, 1/200 rad/s

Check	Result	What it means
Lost to 15 s sampling (half a window)	7.5/200 rad = 2.1°	Zero-order hold
Lost to 5 s reading average	2.5/200 rad = 0.7°	Moving average
Phase margin, realistic	about 84°	Healthy (above 45°)
With 30 s mixing lag as well	about 75°	Still healthy
Delay margin	about 310 s (= λ·π/2)	Pure delay it would take to go unstable

Discrete-time check

T_s = 15 s is λ/13, inside the design-by-emulation rule (T_s < λ/10).

```
G(z) = \frac{K(1-a)}{z-a},\quad a = e^{-15/1711} = 0.991272 \;\Rightarrow\;
G(z) = \frac{2.388}{z - 0.991272} \qquad C(z) = K_p + \frac{K_i T_s}{z-1}
```

Discrete pole	Continuous pole mapped to z = e^(sTs)	Match
0.92537	e^(−15/200) = 0.92774	Close
0.99123	e^(−15/1711) = 0.99127	✓

Both are inside the unit circle, so the digital loop is stable and matches the continuous design closely.

What the linear analysis doesn't show

The 0.1 s pulse quantisation makes the loop hunt in a small limit cycle, the 0.06 pp ripple from section 3. It is bounded and harmless. The simulation captures it; the transfer functions don't.

In MATLAB:

```
K = 273.6; tau = 1711; lambda = 200; Ts = 15;
Kp = tau/(K*lambda); Ki = 1/(K*lambda);
s = tf('s'); G = K/(tau*s + 1);
Gz = c2d(G, Ts, 'zoh'); z = tf('z', Ts);
Tz = feedback((Kp + Ki*Ts/(z - 1))*Gz, 1);
disp(abs(pole(Tz)))                                % all < 1 -> stable
L_real = (Kp + Ki/s)*G/(13*s + 1)*exp(-10*s);    % sensor lag + hold +
averaging
margin(L_real)                                      % about 84 deg
```

8. Building it in Simulink

Build the CO_2 plant from the full nonlinear mass balance (not the linearised transfer function), add a sensor lag, a discrete PI running every 15 s, and a pulse generator that opens the valve for u × 15 s in each window.

Step 1: parameters

```
% ---- CO2 plant ----
V    = 0.99;        % free gas volume, L
Qin  = 0.1/60;      % flow with valve open, L/s
QL   = 5.5e-4;      % leak exchange flow, L/s
xs   = 100;         % supply CO2, %
xa   = 0.04;        % room CO2, %
x0   = 0.04;        % starting CO2, %
tsen = 13;          % ExplorIR-M time constant, s (T90 < 30 s)

% ---- linear model (for tuning) ----
K    = 273.6;       % pp per unit duty
tau  = 1711;        % s

% ---- controller ----
lambda = 200;
Kp   = tau/(K*lambda);   % 0.0313
Ki   = 1/(K*lambda);     % 1.83e-5
Ts   = 15;               % control period, s
tmin = 0.1;              % minimum valve pulse, s
xsp  = 5;                % setpoint, %
band = 0.5;              % integrator freeze band, pp
```

Step 2: the CO_2 plant (nonlinear)

In percent, the mass balance is:

```
\frac{d\chi}{dt} = \frac{v\,Q_{in}(100 - \chi) - Q_L(\chi - 0.04)}{V}
```

where v is the valve state, 0 or 1.

1. **Constant** `xs` minus χ in a **Sum** (`+-`) gives (100 − χ).
2. **Product** block: v × (100 − χ), then **Gain** `Qin`. This product is the nonlinearity, and a Transfer Fcn block can't represent it.
3. χ minus **Constant** `xa` in a **Sum** (`+-`), then **Gain** `QL`.
4. **Sum** (`+-`): inflow minus leak, then **Gain** `1/V`.
5. **Integrator**, Initial condition `x0`. Output is χ, the true chamber CO_2.

Step 3: the sensor

The sensor reads a lagged version of χ. Build the lag the same way as before so it can start at the right value: χ minus the sensor reading into a **Sum** (`+-`), **Gain** `1/tsen`, **Integrator** with Initial condition `x0`. Its output is the measured CO_2.

Step 4: the controller (every 15 s)

1. **Zero-Order Hold**, sample time `Ts`, on the measured CO_2. The controller only sees a new value every 15 s, as on the microcontroller.
2. **Constant** `xsp` minus the held measurement in a **Sum** (`+-`): the error.
3. **P path:** error → **Gain** `Kp`.
4. **I path:** error → **Gain** `Ki` → **Discrete-Time Integrator** (sample time `Ts`, Forward Euler, tick Limit output, 0 to 1).
5. **Sum** (`++`) of P and I → **Saturation** 0 to 1. This is the duty u, updated every 15 s.

No setpoint weighting: b = 1 (section 4).

Adding the integrator freeze (optional, for the lid-opening test). The freeze logic is awkward with plain blocks. The cleanest way is to replace steps 4.2 to 4.5 with one **MATLAB Function** block, run at sample time `Ts` (right-click → Block Parameters → Sample time), that contains exactly the firmware controller from section 9:

```
function u = co2_pi(y, xsp, Kp, Ki, Ts, band)
persistent I reached
if isempty(I), I = 0; reached = false; end
e = xsp - y;
if abs(e) < 0.2, reached = true; end
freeze = reached && abs(e) > band;
u_raw = Kp*e + Ki*I;
u = min(max(u_raw, 0), 1);
if ~freeze && (u == u_raw || (u_raw > 1 && e < 0) || (u_raw < 0 && e > 0))
    I = I + e*Ts;
end
```

Feed `xsp`, `Kp`, `Ki`, `Ts` and `band` in as Constant blocks (or make them parameters). Its output u goes straight to the pulse generator. The upside is that the Simulink controller and the firmware are then literally the same code.

Step 5: the valve pulse generator

This turns u into one pulse per window, like the real firmware:

1. **Gain** `Ts` on u gives the wanted open time in seconds.
2. **Quantizer** with interval `tmin` rounds it to 0.1 s steps.
3. **Repeating Sequence** block: Time values `[0 Ts]`, Output values `[0 Ts]`. This is a sawtooth counting 0 to 15 s in each window.
4. **Relational Operator** `<`: sawtooth < open time. It is true for the first t_on seconds of each window.
5. **Data Type Conversion** to `double`. This is v, the valve state, and it goes into the Product block in step 2.

If you want a quick first version, skip step 5 and feed u straight into the Product block (the average valve opening). It gives almost the same fill curve without the ripple.

Step 6: settings

- Stop time `3600` s (1 h), or `7200` s for the lid-opening test.
- Solver: set **Max step size** to `0.05` (Model Settings → Solver). Otherwise the variable-step solver can step straight over a 0.1 s pulse.
- Scope on χ (true), the sensor reading, and u. Plotting true and measured CO_2 together shows the sensor lag clearly.

Values to expect

Signal	Start	Settles at
χ (true CO_2)	0.04%	5% in about 9 min, peak about 5.06%
u (duty)	about 0.16	about 0.017 (1.7%)
Valve pulses	about 2.3 s per 15 s window	0.2 to 0.3 s per window
Ripple at 5%		about 0.06 pp peak to peak
Sensor reading vs true	lags by about 13 s	

9. Firmware logic

The firmware reads the ExplorIR-M every 0.5 s, runs the PI every 15 s on the average of the last 5 s of readings, fires one valve pulse per window, and wraps it all in safety checks that close the valve on any fault.

Timing

Task	Every	What it does
Read sensor	0.5 s	Read CO_2 over UART, convert ppm → %, store in a 10-reading buffer
Control step	15 s	Average the buffer, run the PI, compute the pulse length, open the valve
Close valve	when the pulse ends	Non-blocking timer, so temperature control keeps running during the pulse
Safety checks	every control step	Close the valve and raise an alarm if anything is wrong

Modes

Mode	When	Valve
Start-up	Power on	Closed. Configure the sensor (auto-zero off), wait for 10 valid readings
Fill	Until the reading first gets within 0.2 pp of 5%	PI with normal integration

Mode	When	Valve
Hold	After that	PI, integrator frozen whenever the error exceeds 0.5 pp (lid openings)
Fault	Any safety check fails	Forced closed until the fault clears and is acknowledged

Fill and Hold are the same PI code; the only difference is the integrator-freeze flag, which switches on once and stays on.

Safety checks

Check	Condition	Action
Sensor not responding	No valid reading for 5 s	Valve closed, alarm
Reading out of range	Below 0 or above 20% (sensor limit)	Valve closed, alarm
CO_2 too high	Above 7%	Valve closed, alarm, integrator held
Pulse cap	u limited to 0.5 (7.5 s per window)	Caps how much gas one bad calculation can add (about 1.2 pp)
Gas supply empty or blocked	Duty above 0.3 for 5 min and CO_2 not rising	Alarm: "check CO_2 supply"

The pulse cap never affects normal operation: the largest duty the PI asks for is about 0.16, at the start of the fill.

The code

```
// ---- settings ----
const float XSP   = 5.0;        // setpoint, %
const float KP    = 0.0313;     // duty per % error
const float KI    = 1.83e-5;    // duty per %.s
const float TS    = 15.0;       // control period, s
const float TMIN  = 0.1;        // minimum valve pulse, s
const float UMAX  = 0.5;        // pulse cap (safety)
const float BAND  = 0.5;        // integrator freeze band, %
const float XHIGH = 7.0;        // over-CO2 alarm, %

// ---- state ----
float I = 0.0;                  // integral, must be a float
```

```
bool  reached = false;          // has the chamber reached 5% yet?
float buf[10]; int nbuf = 0;    // last 5 s of readings

// every 0.5 s
void sensor_task() {
    float ppm;
    if (explorir_read(&ppm)) {                // UART read OK
        push(buf, &nbuf, ppm / 10000.0);      // ppm -> %
        last_good_reading = now();
    }
}

// every 15 s
void co2_control_step() {
    if (fault_active() || nbuf < 10) { valve_close(); return; }

    float y = average(buf, nbuf);             // averaged CO2, %
    float e = XSP - y;
    if (fabs(e) < 0.2) reached = true;        // Fill -> Hold, once

    if (y > XHIGH) { raise_alarm(CO2_HIGH); valve_close(); return; }

    bool freeze = reached && fabs(e) > BAND;  // lid-open protection

    float u_raw = KP*e + KI*I;
    float u = constrain(u_raw, 0.0, UMAX);

    // clamping anti-windup + freeze
    if (!freeze && (u == u_raw || (u_raw > UMAX && e < 0) || (u_raw < 0 && e
> 0))) {
        I += e * TS;
    }

    float t_on = round(u * TS / TMIN) * TMIN;  // quantise to 0.1 s
    if (t_on >= TMIN) valve_open_for(t_on);    // non-blocking, closes itself

    check_supply(u, y);                       // empty-cylinder check
}

// safety, checked continuously
bool fault_active() {
    if (now() - last_good_reading > 5.0) { raise_alarm(SENSOR_LOST); return
true; }
    float y = latest(buf);
```

```
    if (y < 0.0 || y > 20.0)            { raise_alarm(SENSOR_RANGE); return
true; }
    return false;
}
```

`valve_open_for(t)` should switch the solenoid on and start a hardware timer that switches it off after t seconds, so a crash or a slow temperature task can't leave the valve open. For extra safety, add a hardware watchdog that resets the microcontroller (valve off) if the main loop stops running.

Start-up sequence

1. Valve closed.
2. Configure the ExplorIR-M over UART: auto-zero off, check the range is 20%.
3. With the lid open (fresh air), optionally zero the sensor.
4. Close the lid and wait for 10 valid readings (5 s).
5. Start the PI in Fill mode. No separate burst: the PI does the fill (about 9 min).

10. Once it is built

The leak test and fill test from your write-up give Q_L and Q_in, and from those every model number and both controller gains follow; the same tests also measure the sensor lag.

Test 1: leak (decay) test, the most important one

Charge the chamber to about 5%, close the valve and log the sensor for about an hour. Fit:

```
\chi_c(t) = \chi_a + (\chi_0 - \chi_a)\,e^{-t/\tau_L}, \qquad Q_L = \frac{V}
{\tau_L}
```

Plot ln(χ_c − χ_a) against time: it should be a straight line with slope −1/τ_L. A curved line means the chamber isn't well mixed or the leak changes with time (for example a lid that seals unevenly). Do it with the heaters running at 37 °C, because warm gas and a warm printed wall can leak differently.

Test 2: fill test

From a known steady reading, open the valve for a fixed time (about 10 s), close it, and wait for the reading to settle.

```
Q_{in} = \frac{V}{t}\ln\left(\frac{\chi_s - \chi_0}{\chi_s - \chi_1}\right)
```

Two things to watch:

- **Wait for the sensor, but correct for the leak.** The sensor takes a few minutes to settle, and over 5 min the leak removes about 15% of the added CO_2 (e^(−300/1800) = 0.85). Either extrapolate the decay curve back to the moment the valve closed, or use the Q_L from test 1 to correct it.
- **The same data gives the sensor lag.** The true CO_2 jumped in 10 s; the time the sensor takes to show 63% of the rise is its time constant. Use it to check λ (section 5).

Test 3: minimum pulse

Repeat the fill test with 0.05, 0.1, 0.2 and 0.5 s pulses. The shortest pulse that gives a consistent rise is your real t_min. If it is longer than 0.1 s, increase T_s so the duty resolution t_min/T_s stays below about 0.5%.

Update the model

```
\mu_0 = \frac{Q_L(\chi_0 - \chi_a)}{Q_{in}(\chi_s - \chi_0)} \qquad \tau = \frac{V}{\mu_0 Q_{in} + Q_L} \qquad K = \frac{100\,Q_{in}(\chi_s - \chi_0)}{\mu_0 Q_{in} + Q_L}
```

Retune

```
K_p = \frac{\tau}{K(\lambda + \theta)} \qquad K_i = \frac{1}{K(\lambda + \theta)}
```

where θ is any pure delay you saw in the fill test (tubing, sensor). Keep λ at least 4 to 5 times the sensor time constant, and keep T_s ≤ λ/10.

Check the integrator freeze on the real chamber

Open the lid for 10 s and 60 s after the chamber has settled, and log the recovery. It should come back to 5% in under 10 min with less than 0.1 to 0.15 pp overshoot. If the overshoot is bigger, widen the freeze band slightly (0.5 → 0.7 pp); if it stalls short of 5%, narrow it.

What changes and what doesn't

Stays the same	Gets updated
PI structure, b = 1, clamping anti-windup	Q_L, Q_in, and so μ_0, τ, K
15 s time-proportioned pulses, integrator freeze	Kp, Ki
Nonlinear Simulink plant and test plan	T_s, if t_min turns out longer

For the report: model, why the PI does the fill (sensor lag vs 30 s fill time), lambda tuning, pulse scheme, simulated fill, lid opening and robustness, leak and fill tests, retuned gains.